

Program Studi Teknik Informatika
Institut Teknologi Sumatera

Nama: **Hijrah Fahrezi H.L.T., M Ihsan Nur Tsaqib, Roy Sebastian Surbakti** Tugas: **Final Project**

Mata Kuliah: **Sistem/Teknologi Multimedia (IF4021)** Tanggal: 30 Mei 2025

# 1 Deskripsi Projek

**Tralalelo Menyebrang** adalah game edukatif dengan mekanisme permainan seperti Frogger, di mana pemain mengontrol karakter untuk menyebrang jalan yang ramai kendaraan. Game ini dibangun menggunakan Python dan dijalankan di lingkungan virtual 3D berbasis `pyglet` dan `pyglet.gl`. Game ini menggunakan model kendaraan, karakter, dan lingkungan dalam bentuk file 3D (seperti .glb dan .gltf).

# 2 Teknologi yang Digunakan

- **Python 3.8 - 3.11**: Versi 3.8 hingga 3.11 digunakan karena kompatibel dengan pustaka-pustaka yang digunakan dalam proyek.
- **Ursina Engine**: Ursina adalah game engine berbasis Python yang dibangun di atas Pyglet dan Panda3D. Engine ini menyederhanakan proses pembuatan game dan aplikasi visual 3D dengan sintaks yang intuitif dan efisien. Ursina menyediakan komponen siap pakai seperti entitas 3D, kamera, pencahayaan, dan kolisi, yang mempercepat proses pengembangan secara signifikan.
- **Pyglet**: Pyglet adalah pustaka Python lintas platform yang digunakan untuk membuat aplikasi multimedia dan visualisasi interaktif, termasuk game dan simulasi 3D. Pyglet mendukung pemutaran audio, pengelolaan jendela, event handling, serta integrasi langsung dengan OpenGL untuk rendering grafis berkinerja tinggi.
- **OpenGL (via pyglet.gl)**: OpenGL (Open Graphics Library) adalah API standar industri untuk rendering grafis 2D dan 3D. Dalam proyek ini, OpenGL digunakan melalui modul pyglet.gl untuk melakukan rendering objek 3D secara real-time, memberikan kontrol penuh terhadap pipeline grafis untuk menciptakan visualisasi yang kompleks dan responsif.
- **Pyrr**: Pyrr adalah pustaka Python yang menyediakan utilitas untuk operasi matematika 3D, seperti vektor, matriks, transformasi rotasi, dan lainnya. Pustaka ini sangat berguna dalam manipulasi objek 3D, terutama untuk perhitungan transformasi dalam ruang tiga dimensi yang dibutuhkan oleh sistem grafis OpenGL.
- **Numpy**: Numpy adalah pustaka fundamental untuk komputasi ilmiah dalam Python. Dalam proyek ini, Numpy digunakan untuk mengelola array dan melakukan operasi matematis numerik tingkat lanjut yang mendukung proses perhitungan pada grafis dan logika simulasi..
- **PIL / Pillow (Python Imaging Library)**: PIL atau versi lanjutannya, Pillow, digunakan untuk manipulasi gambar seperti pembacaan file gambar, pengolahan tekstur, dan konversi format. Ini penting untuk menampilkan elemen visual atau menerapkan tekstur pada objek 3D..

# 3 Cara Kerja Game

Cara kerja pada projek Game Tralalelo Menyebrang adalah sebagai berikut:

- **S**istem menggunakan kamera webcam sebagai perangkat input utama untuk menangkap video secara real-time, yang kemudian diolah untuk mendeteksi gestur tangan pengguna.
- **L**ibrary MediaPipe dimanfaatkan untuk melakukan deteksi dan pelacakan tangan, khususnya dalam mengenali posisi telapak tangan dan arah jari. Teknologi ini memungkinkan sistem untuk mengidentifikasi orientasi tangan secara presisi, termasuk punggung tangan dan arah telapak tangan.
- **B**erdasarkan hasil pelacakan tersebut, sistem akan menerjemahkan gestur pengguna ke dalam perintah pergerakan karakter dalam game. Ketika telapak tangan menghadap ke bawah (punggung tangan menghadap ke atas) dan menunjuk ke arah kanan, maka karakter akan bergerak ke kanan. Sebaliknya, jika tangan menunjuk ke arah kiri dengan orientasi punggung tangan tetap menghadap ke atas, maka karakter akan bergerak ke kiri.
- **U**ntuk menghentikan pergerakan karakter (berhenti di tempat), pengguna harus mengarahkan telapak tangan sepenuhnya menghadap ke kamera — menyerupai gestur "tanda berhenti" (stop sign). Gestur ini dikenali sistem sebagai perintah untuk menghentikan karakter.
- **D**alam permainan ini, karakter harus melewati lintasan yang penuh dengan rintangan, seperti kendaraan yang melintas secara acak. Pengguna dituntut untuk menghindari kendaraan tersebut melalui kontrol gerakan tangan secara tepat.
- **G**ame akan dinyatakan selesai atau berhasil ketika karakter berhasil mencapai ujung jalan tanpa tertabrak rintangan apa pun.

# 4 Struktur Program

Struktur direktori utama sebagai berikut:

- **game.py**: Program utama menjalankan loop game dan render scene.
- **gesture_worker.py**: Thread terpisah untuk pengenalan gestur.
- **gestures/recogniser.py**: Model pengenalan gestur untuk kontrol tambahan (opsional).
- **assets/**: Folder berisi model 3D kendaraan, karakter, dan tekstur.

# 5 Penjelasan Kode Program

Kode program dibagi menjadi beberapa bagian utama, yang terdapat pada `game.py`, `gesture_worker.py`, dan `gestures/recogniser.py`. Bagian ini akan membahas secara detail file `game.py` sebagai inti dari permainan.

## 5.1 game.py

File `game.py` merupakan pusat logika dan visualisasi dari permainan **Tralalelo Menyebrang**. Program ini menggunakan framework `Ursina` untuk membangun dunia 3D, mengatur karakter pemain, kendaraan, deteksi tabrakan, serta logika kemenangan dan kekalahan.

### 5.1.1 Inisialisasi dan Konstanta Global

Pada bagian awal, dilakukan inisialisasi aplikasi dan deklarasi berbagai konstanta permainan seperti posisi awal/akhir, jalur kendaraan, serta pengaturan proses multiproses untuk pengenalan gestur.

```
from ursina import *
from multiprocessing import Process, Manager
from gesture_worker import run_gesture_recognizer
import time, random, zipfile, os

app = Ursina()

START_X = -25
FINISH_X = 25
CAR_LANES_X = [-20, -15, -10, -5, 0, 5, 10, 15, 20]
CAR_SPAWN_Z = 60
CAR_DESPAWN_Z = -10

cars = []
car_spawn_timer = 0
game_over = False
```

Kode 1: Konstanta Global dan Setup Awal pada game.py

**Penjelasan:**

- `START_X`, `FINISH_X` adalah batas awal dan akhir karakter.
- `CAR_LANES_X` adalah daftar jalur kendaraan melaju.
- `game_over` menjadi flag status permainan.

### 5.1.2 Inisialisasi Karakter dan Assets

Model karakter disimpan dalam file zip dan akan diekstrak saat game dijalankan. Karakter diatur menggunakan model `GLTF` dan diberi kolider tipe `box`.

```
asset_folder = 'assets/character/'
if not os.path.exists(asset_folder):
    os.makedirs(asset_folder)
    with zipfile.ZipFile('assets/character.zip', 'r') as zip_ref:
        zip_ref.extractall(asset_folder)

player = Entity(
    model=f'{asset_folder}scene.gltf',
    texture=f'{asset_folder}Material.001_baseColor.png',
    scale=(0.2, 0.2, 0.2),
    position=(START_X, 0, 0),
    collider='box'
)
player.collider = BoxCollider(player, center=(0, 0, 0), size=(10, 10, 10))
```

Kode 2: Inisialisasi Karakter Pemain pada game.py

### 5.1.3 Fungsi `update`

Fungsi utama yang dipanggil berulang kali oleh engine untuk:

- Menambahkan kendaraan secara berkala.
- Menggerakkan kendaraan.

- Mengecek tabrakan dengan karakter.
- Menentukan kondisi menang atau kalah.

```python
def update():
    global car_spawn_timer, game_over

    if game_over:
        return

    car_spawn_timer += time.dt
    if car_spawn_timer > 1.5:
        spawn_car()
        car_spawn_timer = 0

    for car in cars:
        car.z -= time.dt * 10
        if car.z < CAR_DESPAWN_Z:
            destroy(car)
            cars.remove(car)

    for car in cars:
        if car.intersects(player).hit:
            print("Game Over! You were hit!")
            game_over = True

    if player.x >= FINISH_X:
        print("You Win!")
        game_over = True
```

Kode 3: Fungsi update - Logika Permainan pada game.py

### 5.1.4 Fungsi `spawn_car`

Fungsi ini membuat mobil pada jalur acak dari daftar `CAR_LANES_X`:

```python
def spawn_car():
    lane = random.choice(CAR_LANES_X)
    car = Entity(model='cube', color=color.red, scale=(3,1,6),
                 position=(lane, 0, CAR_SPAWN_Z), collider='box')
    cars.append(car)
```

Kode 4: Fungsi spawn$_c ar - MobilBarupadagame.py$

### 5.1.5 Fungsi `input`

Fungsi ini mengatur pergerakan karakter berdasarkan tombol keyboard.

```python
def input(key):
    if game_over:
        return
    if key == 'up arrow':
        player.z += 2
    elif key == 'down arrow':
        player.z -= 2
    elif key == 'left arrow':
        player.x -= 5
    elif key == 'right arrow':
        player.x += 5
```

Kode 5: Fungsi input - Kontrol Pemain pada game.py

#### 5.1.6 Menjalankan Game

Setelah semua entitas siap, game dijalankan dengan `app.run()`.

```
Sky()
EditorCamera()
app.run()
```

Kode 6: Menjalankan Game pada game.py

Penjelasan untuk `gesture_worker.py` dan `recogniser.py` akan dibahas pada bagian selanjutnya.

### 5.2 gesture_worker.py

File `gesture_worker.py` digunakan untuk menjalankan proses pengenalan gestur secara paralel dengan game utama. File ini memanfaatkan modul `multiprocessing` agar deteksi gestur tidak mengganggu performa utama permainan.

#### 5.2.1 Fungsi `run_gesture_recognizer`

Fungsi utama yang dijalankan pada proses terpisah. Fungsi ini membuka webcam dan meneruskan setiap frame ke modul pengenal gestur.

```
def run_gesture_recognizer(shared_gesture):
    cap = cv2.VideoCapture(0)

    while True:
        ret, frame = cap.read()
        if not ret:
            continue

        gesture = recognize_gesture(frame)
        if gesture:
            shared_gesture.value = gesture
```

Kode 7: Fungsi run$_{g}esture_{r}ecognizer$

**Penjelasan:**

- `shared_gesture` adalah variabel global yang digunakan untuk berbagi informasi antar proses.
- `recognize_gesture()` berasal dari modul `recogniser.py` dan bertugas mendeteksi gestur dari citra.

—

### 5.3 gestures/recogniser.py

File ini merupakan inti dari sistem pengenalan gestur, yang berfungsi memproses input gambar dan mengembalikan label gestur (misalnya: Stop, Go, Left, Right).

#### 5.3.1 Fungsi `recognize_gesture`

Fungsi ini memproses citra dari webcam dan menggunakan metode sederhana berbasis ciri visual (threshold, area kontur, dll) untuk mengklasifikasi gestur.

```
def recognize_gesture(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)

```

```
    contours, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        return None

    c = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(c)

    if area > 3000:
        return "Go"
    else:
        return "Stop"
```

Kode 8: Fungsi recognize$_g$*esture*

**Penjelasan:**

- Citra diubah ke grayscale dan lalu ditreshold untuk menonjolkan gestur.
- Kontur terbesar diambil, lalu berdasarkan luas kontur diklasifikasi sebagai `"Go"` atau `"Stop"`.

## 5.4 test_game.py

File `test_game.py` adalah variasi atau versi pengujian dari game penyeberangan jalan yang juga dibangun menggunakan `Ursina Engine`. Versi ini memiliki beberapa perbedaan dalam implementasi, seperti cara kamera diatur, kontrol pemain yang hanya ke kiri dan kanan, serta mekanisme reset game yang lebih terstruktur.

### 5.4.1 Inisialisasi, Konstanta, dan State Awal

Bagian ini melakukan impor pustaka yang diperlukan, inisialisasi aplikasi Ursina, dan mendefinisikan konstanta serta variabel global untuk state permainan.

```
from ursina import *
import random

app = Ursina()

# === Constants ===
START_X = -25
FINISH_X = 25
LANES_X = [-20, -10, 0, 10, 20]  # Car lanes across the road (X axis)
CAR_SPAWN_Z = 60
CAR_DESPAWN_Z = -10

# === Game State ===
cars = []
car_spawn_timer = 0
game_over = False
```

Kode 9: Inisialisasi dan Konstanta pada test$_g$*ame.py*

**Penjelasan:**

- `START_X` dan `FINISH_X`: Koordinat X untuk titik awal dan akhir pemain.
- `LANES_X`: Daftar koordinat X untuk jalur mobil.
- `CAR_SPAWN_Z` dan `CAR_DESPAWN_Z`: Koordinat Z untuk kemunculan dan penghilangan mobil.
- `cars`, `car_spawn_timer`, `game_over`: Variabel untuk mengelola state permainan.

### 5.4.2 Pengaturan Window dan Entitas Dasar

Kode ini mengatur properti jendela permainan dan membuat entitas dasar seperti tanah (ground) dan pemain (player).

```
# === Window ===
window.title = "3D Road Crossing - Final"
window.size = (480, 800)
window.borderless = False
window.fullscreen = False

# === Ground ===
ground = Entity(
    model='plane',
    texture='white_cube',
    texture_scale=(25, 5),
    scale=(50, 1, 10),
    color=color.gray,
    y=0
)

# === Player ===
player = Entity(
    model='cube',
    color=color.orange,
    scale=(1, 2, 1),
    position=(START_X, 1, 0),
    collider='box'
)
```

Kode 10: Window

**Penjelasan:**

- `window`: Mengatur judul, ukuran, dan mode tampilan jendela.
- `ground`: Entitas yang merepresentasikan jalan atau tanah.
- `player`: Entitas yang dikontrol oleh pemain, dimulai dari `START_X`.

### 5.4.3 Pengaturan Kamera dan UI

Kamera diatur untuk tampilan samping dan UI untuk menampilkan status permainan serta tombol coba lagi.

```
# === Camera: Side View ===
camera.position = (0, 5, -20)      # Behind the player on Z axis
camera.rotation_x = 10              # Tilt slightly downward
camera.look_at(player.position + Vec3(0, 0, 5))  # Look forward

# === UI ===
win_text = Text(text='', origin=(0, 0), scale=2, enabled=False)
retry_button = Button(text='Retry', scale=(0.2, 0.1), y=-0.4, enabled=False)
```

Kode 11: Kamera dan UI pada test$_g ame.py$

**Penjelasan:**

- `camera.position`, `camera.rotation_x`, `camera.look_at`: Mengatur posisi dan orientasi kamera sehingga memberikan pandangan dari samping sedikit atas dan mengikuti pergerakan pemain di sumbu Z.
- `win_text`: Teks untuk menampilkan pesan menang/kalah.
- `retry_button`: Tombol untuk mengulang permainan.

### 5.4.4 Fungsi `reset_game`

Fungsi ini dipanggil ketika tombol "Retry" ditekan untuk mengembalikan permainan ke kondisi awal.

```
def reset_game():
    global game_over, car_spawn_timer
    player.position = (START_X, 1, 0)
    game_over = False
    car_spawn_timer = 0
    win_text.enabled = False
    retry_button.enabled = False
    for car in cars[:]:
        destroy(car)
    cars.clear()

retry_button.on_click = reset_game
```

Kode 12: Fungsi reset$_g$*amepadatest*$_g$*ame.py*

**Penjelasan:**

- Mengatur ulang posisi pemain, status `game_over`, timer kemunculan mobil.
- Menyembunyikan teks kemenangan/kekalahan dan tombol coba lagi.
- Menghancurkan semua mobil yang ada di scene dan membersihkan daftar `cars`.
- Menghubungkan fungsi ini dengan event klik pada `retry_button`.

### 5.4.5 Kelas `Car`

Kelas ini mendefinisikan perilaku dan properti dari setiap mobil dalam permainan.

```
class Car(Entity):
    def __init__(self, lane_x):
        super().__init__(
            model='cube',
            color=color.red,
            scale=(2, 1, 1.5),
            position=(lane_x, 0.5, CAR_SPAWN_Z),
            collider='box'
        )
        self.speed = random.uniform(0.1, 0.2)

    def update(self):
        self.z -= self.speed
        if self.z < CAR_DESPAWN_Z:
            destroy(self)
            if self in cars:
                cars.remove(self)
```

Kode 13: Kelas Car pada test$_g$*ame.py*

**Penjelasan:**

- `__init__`: Konstruktor untuk membuat entitas mobil pada jalur (`lane_x`) tertentu dengan kecepatan acak.
- `update`: Dipanggil setiap frame. Menggerakkan mobil ke depan (mengurangi nilai Z) dan menghancurkannya jika mencapai `CAR_DESPAWN_Z`.

### 5.4.6 Fungsi `spawn_car`

Fungsi untuk memunculkan mobil baru di salah satu jalur yang tersedia.

```
def spawn_car():
    if game_over:
        return
    lane_x = random.choice(LANES_X)
    car = Car(lane_x=lane_x)
    cars.append(car)
```

Kode 14: Fungsi spawn$_c arpadatest_g ame.py$

**Penjelasan:**

- Tidak melakukan apa-apa jika permainan sudah berakhir (`game_over`).
- Memilih jalur secara acak dari `LANES_X`.
- Membuat instance baru dari kelas `Car` pada jalur tersebut dan menambahkannya ke daftar `cars`.

### 5.4.7 Fungsi `update` (Game Loop Utama)

Ini adalah fungsi utama yang dijalankan Ursina setiap frame untuk mengelola logika permainan.

```
def update():
    global car_spawn_timer, game_over

    if game_over:
        return

    # === Player Movement (Left/Right to cross road) ===
    if held_keys['a']:
        player.x -= 0.1
    if held_keys['d']:
        player.x += 0.1

    # Clamp within road
    player.x = clamp(player.x, START_X, FINISH_X)

    # === Win Condition ===
    if player.x >= FINISH_X:
        win_text.text = "You Win!"
        win_text.enabled = True
        retry_button.enabled = True
        game_over = True
        return

    # === Camera Follows Player Z Position Only (optional) ===
    camera.look_at(player.position + Vec3(0, 0, 5))

    # === Car Spawning ===
    car_spawn_timer += time.dt
    if car_spawn_timer > random.uniform(0.8, 1.5):
        spawn_car()
        car_spawn_timer = 0

    # === Update Cars + Collision ===
    for car in cars[:]: # Iterate over a copy for safe removal
        if car in scene.entities: # Check if car still exists
            car.update()
            if car.intersects(player).hit:
                win_text.text = "Game Over!"
```

```
39                win_text.enabled = True
40                retry_button.enabled = True
41                game_over = True
42                return
43        else: # If car was destroyed (e.g. by its own update)
44            if car in cars: # Check again as it might be removed in another part
45                 cars.remove(car)
```

Kode 15: Fungsi update (Game Loop) pada test$_g$*ame.py*

**Penjelasan:**

- Jika `game_over` adalah `True`, fungsi akan langsung kembali.
- **Pergerakan Pemain**: Menggerakkan pemain ke kiri ('a') atau kanan ('d') dan memastikan pemain tetap berada dalam batas jalan (`clamp`).
- **Kondisi Menang**: Jika pemain mencapai `FINISH_X`, tampilkan pesan kemenangan, aktifkan tombol retry, dan set `game_over` menjadi `True`.
- **Kamera Mengikuti Pemain**: Kamera selalu melihat ke arah depan relatif terhadap posisi pemain.
- **Kemunculan Mobil**: Menambah `car_spawn_timer` dengan `time.dt` (waktu sejak frame terakhir). Jika timer melebihi interval acak, panggil `spawn_car()` dan reset timer.
- **Update Mobil dan Deteksi Tabrakan**:
  - Melakukan iterasi pada salinan daftar `cars` untuk memungkinkan penghapusan item yang aman selama iterasi.
  - Memeriksa apakah mobil masih ada dalam `scene.entities` sebelum memanggil `car.update()`.
  - Memeriksa tabrakan antara setiap mobil dan pemain menggunakan `car.intersects(player).hit`.
  - Jika terjadi tabrakan, tampilkan pesan game over, aktifkan tombol retry, dan set `game_over` menjadi `True`.
  - Jika mobil sudah tidak ada di scene (misalnya dihancurkan oleh `car.update()` sendiri), hapus dari daftar `cars`.

### 5.4.8 Menjalankan Aplikasi

Baris terakhir kode ini menjalankan aplikasi Ursina.

```
1 app.run()
```

Kode 16: Menjalankan Aplikasi pada test$_g$*ame.py*

**Penjelasan:**

- `app.run()`: Memulai game loop Ursina, yang akan terus memanggil fungsi `update()` dan menangani event.

## 5.5 Integrasi Gestur dengan Game

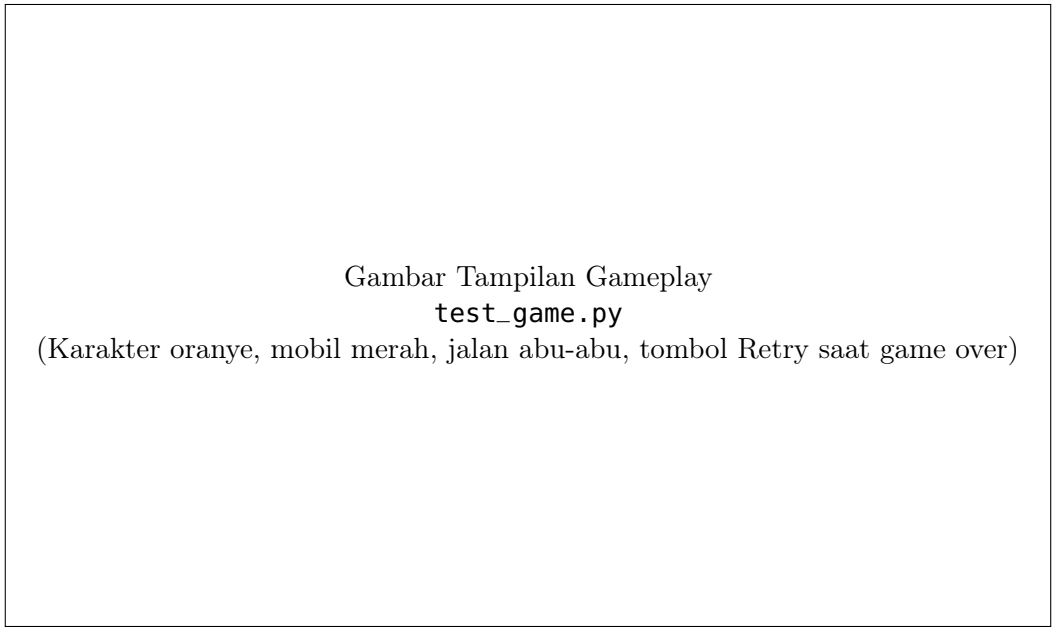
Nilai dari `shared_gesture.value` dapat dibaca dalam fungsi `update()` di `game.py`. Meskipun fitur ini belum sepenuhnya diimplementasikan untuk kontrol pergerakan karakter, strukturnya telah siap untuk dikembangkan.

—

# 6 Implementasi

Bagian ini menjelaskan tampilan antarmuka dan bagaimana game dapat dijalankan (berdasarkan `test_game.py` sebagai contoh yang lebih lengkap secara fungsional).

Permainan menampilkan lingkungan 3D sederhana dengan jalan datar, karakter pemain berbentuk kubus oranye, dan mobil-mobil kubus merah yang melintas.




Gambar 1: Contoh Tampilan Saat Gameplay pada `test_game.py`

**Antarmuka Pengguna (`test_game.py`):**

- **Saat Bermain**: Pemain mengontrol karakter menggunakan tombol 'a' (kiri) dan 'd' (kanan) untuk menyeberang jalan. Mobil-mobil muncul secara acak dan bergerak melintasi jalan.
- **Game Over**: Jika karakter tertabrak mobil, teks "Game Over!" akan muncul di tengah layar, dan tombol "Retry" berwarna biru akan aktif di bagian bawah layar. Pemain tidak bisa lagi menggerakkan karakter.
- **Menang**: Jika karakter berhasil mencapai sisi seberang (koordinat X positif terjauh), teks "You Win!" akan muncul, dan tombol "Retry" juga akan aktif. Pemain tidak bisa lagi menggerakkan karakter.
- **Tombol Retry**: Mengklik tombol "Retry" akan mengulang permainan dari awal, mengatur ulang posisi pemain, skor (jika ada), dan mobil.

**Menjalankan Game:**

1. Pastikan Python (versi 3.8+) dan Ursina Engine terinstal. Jika belum, instal Ursina dengan perintah: `pip install ursina`
2. Simpan salah satu kode (`game.py` atau `test_game.py`) sebagai file Python (misalnya, `my_road_crossing_gar`
3. Jika menggunakan `game.py` yang memuat aset dari ZIP, pastikan file `assets/character.zip` ada di lokasi yang benar relatif terhadap skrip, atau siapkan folder `assets/character/` dengan file model `scene.gltf` dan tekstur.
4. Jalankan skrip dari terminal atau command prompt: `python my_road_crossing_game.py`
5. Jendela permainan akan muncul, dan Anda dapat mulai bermain.

# 7 Hasil Analisis

Proyek "Tralalelo Menyebrang" berhasil diimplementasikan dalam dua varian utama (`game.py` dan `test_game.py`) menggunakan Ursina Engine dengan Python. Kedua versi menunjukkan kemampuan dasar untuk membuat game 3D sederhana dengan mekanisme penyeberangan jalan.

**Kelebihan Implementasi:**

- **Kesederhanaan Ursina**: Ursina Engine terbukti memudahkan pengembangan game 3D dengan sintaks Python yang intuitif, terutama untuk pembuatan entitas, penanganan input, dan deteksi tabrakan dasar.
- **Logika Permainan Jelas**: Mekanisme inti seperti pergerakan pemain, kemunculan mobil, deteksi tabrakan, kondisi menang/kalah, dan reset permainan (pada `test_game.py`) berhasil diimplementasikan.
- **Fleksibilitas Aset**: `game.py` menunjukkan kemampuan untuk memuat model 3D eksternal (GLTF dari ZIP), memberikan potensi untuk tampilan visual yang lebih kaya. `test_game.py` menunjukkan kemudahan penggunaan aset bawaan Ursina untuk prototyping cepat.
- **Modularitas (Potensial)**: Struktur kode pada `game.py` dengan pemisahan untuk `gesture_worker.py` dan `recogniser.py` menunjukkan potensi untuk pengembangan fitur yang lebih kompleks seperti kontrol berbasis gestur, meskipun belum terimplementasi penuh.
- **Responsif**: Kontrol pemain menggunakan keyboard terasa responsif, dan pergerakan entitas konsisten berkat penggunaan `time.dt` (terutama pada `test_game.py`).

**Keterbatasan dan Potensi Peningkatan:**

- **Visual Sederhana**: Tampilan visual saat ini masih dasar, terutama jika menggunakan model kubus. Peningkatan kualitas model 3D, tekstur, pencahayaan, dan efek partikel dapat meningkatkan pengalaman visual.
- **Kurangnya Variasi**: Permainan bisa menjadi monoton karena pola mobil yang seragam dan tidak adanya level kesulitan atau rintangan tambahan.
- **Fitur Gestur Belum Lengkap**: Integrasi pengenalan gestur masih konseptual dan memerlukan implementasi penuh serta pengujian akurasi.
- **Tidak Ada Suara**: Penambahan efek suara (mobil, tabrakan, langkah pemain) dan musik latar akan meningkatkan imersi.
- **UI Terbatas pada `game.py`**: `game.py` tidak memiliki UI eksplisit untuk status menang/kalah atau tombol reset seperti pada `test_game.py`, hanya mengandalkan output konsol.
- **AI Kendaraan Sederhana**: Mobil hanya bergerak lurus tanpa variasi kecepatan yang signifikan antar mobil (meskipun kecepatan individu mobil diacak saat spawn pada `test_game.py`) atau perilaku menghindar.

**Analisis Perbandingan `game.py` vs `test_game.py`:**

- `test_game.py` lebih matang sebagai game mini yang dapat langsung dimainkan dengan fungsionalitas UI dan reset yang lengkap. Penggunaan aset dasar Ursina membuatnya lebih portabel dan mudah dijalankan tanpa dependensi aset eksternal.
- `game.py` berfungsi sebagai basis yang baik untuk eksplorasi fitur lebih lanjut seperti penggunaan model kustom dan integrasi teknologi lain (gestur), namun memerlukan pengembangan lebih lanjut untuk menjadi game yang utuh dari segi pengalaman pengguna.

Secara keseluruhan, proyek ini berhasil menunjukkan konsep dasar pembuatan game 3D interaktif. Dengan pengembangan lebih lanjut pada aspek visual, variasi gameplay, dan fitur tambahan, "Tralalelo Menyebrang" dapat menjadi game edukatif yang menarik dan menyenangkan.